

ASSESSMENT TOOL-CO2-AT2

1. CONSULTATION CHARGE SYSTEM

Code:

```
#include <iostream>
#include <string>
using namespace std;

// Function prototyping
float calculateCharge(string patientType, float baseCharge, float serviceCharge = 100.0);
float calculateCharge(string patientType, float baseCharge, float specialistFee, float
serviceCharge);
void modifyBill(float &finalBill, float additionalCharge);
void displayDetails(string patientType, float totalCharge);

int main() {
    string patientType;
    float baseCharge, specialistFee, finalBill, additionalCharge;
    int choice;

    cout << "==== Hospital Consultation Charge System =====>> endl;
    cout << "1. General Consultation">> endl;
    cout << "2. Specialist Consultation">> endl;
    cout << "Enter your choice: ";
    cin >> choice;

    if (choice == 1) {
        patientType = "General";
        cout << "Enter base consultation charge: ";
        cin >> baseCharge;

        // Function overloading – version with default service charge
        finalBill = calculateCharge(patientType, baseCharge);
    }
    else if (choice == 2) {
        patientType = "Specialist";
        cout << "Enter base consultation charge: ";
        cin >> baseCharge;
        cout << "Enter specialist fee: ";
        cin >> specialistFee;
```

```

float customServiceCharge;
cout << "Enter service charge (or -1 to use default): ";
cin >> customServiceCharge;

if (customServiceCharge == -1)
    finalBill = calculateCharge(patientType, baseCharge, specialistFee, 100.0);
else
    finalBill = calculateCharge(patientType, baseCharge, specialistFee, customServiceCharge);
}
else {
    cout << "Invalid choice!" << endl;
    return 0;
}

// Call by reference to modify final bill
cout << "Enter any additional charge to modify bill (0 if none): ";
cin >> additionalCharge;
modifyBill(finalBill, additionalCharge);

// Display final result
displayDetails(patientType, finalBill);

return 0;
}

// Function overloading – General Consultation (with default service charge)
float calculateCharge(string patientType, float baseCharge, float serviceCharge) {
    return baseCharge + serviceCharge;
}

// Function overloading – Specialist Consultation
float calculateCharge(string patientType, float baseCharge, float specialistFee, float
serviceCharge) {
    return baseCharge + specialistFee + serviceCharge;
}

// Call by reference to modify final bill amount
void modifyBill(float &finalBill, float additionalCharge) {
    finalBill += additionalCharge;
}

// Display patient type and total consultation charge

```

```

void displayDetails(string patientType, float totalCharge) {
cout << "\n===== Consultation Bill Summary =====> << endl;
cout << "Patient Type: " << patientType << endl;
cout << "Total Consultation Charge: Rs. " << totalCharge << endl;

}

```

Output:

PROGRAM EDITOR

C++

Run

Save

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 // Function Prototyping
6 float calculateCharge(string patientType, float baseCharge, float serviceCharge = 100.
7 float calculateCharge(string patientType, float baseCharge, float specialistFee, float
8 void applyServiceCharge(float &finalBill, float serviceCharge);
9 void displayDetails(string patientType, float totalCharge);
10
11 // Function Overloading - General Consultation
12 float calculateCharge(string patientType, float baseCharge, float serviceCharge) {
13     float total = baseCharge + serviceCharge;
14     return total;
15 }
16

```

INPUT

```

1
500
50

```

```

===== Hospital Consultation Charge System ====
1. General Consultation
2. Specialist Consultation
Enter your choice: Enter base consultation charge: Do you want to add an extra
standard service charge of Rs.50? (y/n):
Patient Type: General Patient
Total Consultation Charge: Rs. 600

```

2. FITNESS CENTRE MEMBERSHIP SYSTEM

Code:

```

#include <iostream>
#include <string>

```

```

#include <iomanip>
using namespace std;

// ---- Pricing constants (easy to modify) ----
const double BASIC_FEE = 999.0;
const double SILVER_FEE = 1499.0;
const double GOLD_FEE = 1999.0;

class Member {
private:
    int memberId;
    string name;
    string planType;
    double fee;

    static int activeMembers; // total active members

    // Validate membership plan
    bool isValidPlan(const string& plan) {
        return (plan == "BASIC" || plan == "SILVER" || plan == "GOLD");
    }

public:
    // Constructor
    Member() {
        memberId = 0;
        name = "";
        planType = "";
        fee = 0.0;
    }

    // Input member details
    void assignDetails(int id, const string& nm, const string& plan) {
        memberId = id;
        name = nm;

        // Plan validation
        if (isValidPlan(plan)) {
            planType = plan;
            calculateFee();
        } else {
            planType = "INVALID";
            fee = 0.0;
        }
    }
};

```

```
}  
}
```

```
// Assign plan (with validation)  
void assignPlan(const string& plan) {  
    if (isValidPlan(plan)) {  
        planType = plan;  
        calculateFee();  
    } else {  
        planType = "INVALID";  
        fee = 0.0;  
    }  
}
```

```
// Calculate fee based on plan type  
void calculateFee() {  
    if (planType == "BASIC") fee = BASIC_FEE;  
    else if (planType == "SILVER") fee = SILVER_FEE;  
    else if (planType == "GOLD") fee = GOLD_FEE;  
    else fee = 0.0;  
}
```

```
// Display membership details  
void display() const {  
    cout << left  
    << setw(10) << memberId  
    << setw(18) << name  
    << setw(12) << planType  
    << "Fee: " << fee << endl;  
}
```

```
// Update active members count when member is valid  
void activateIfValid() {  
    if (planType != "INVALID") {  
        activeMembers++;  
    }  
}
```

```
// Static function to show total active members  
static void showActiveMembers() {  
    cout << "Total Active Members: " << activeMembers << endl;  
}  
};
```

```

// Static member definition
int Member::activeMembers = 0;

int main() {
    int n;
    cout << "Enter number of members: ";
    cin >> n;

    // Array of objects
    Member members[50];

    for (int i = 0; i < n; i++) {
        int id;
        string nm, plan;

        cout << "\n--- Member " << (i + 1) << " ---\n";
        cout << "Enter Member ID: ";
        cin >> id;
        cout << "Enter Name: ";
        cin.ignore();
        getline(cin, nm);
        cout << "Enter Plan Type (BASIC/SILVER/GOLD): ";
        cin >> plan;

        members[i].assignDetails(id, nm, plan);
        members[i].activateIfValid(); // count only valid plans
    }

    cout << "\n==== Membership Details =====\n";
    cout << left << setw(10) << "ID"
    << setw(18) << "Name"
    << setw(12) << "Plan"
    << "Fee" << endl;

    cout << "-----\n";

    for (int i = 0; i < n; i++) {
        members[i].display();
    }

    cout << "\n";
    Member::showActiveMembers();
}

```

```
return 0;
}
```

Output:

PROGRAM EDITOR

C++

Run

Save

```
1 #include <iostream>
2 #include <string>
3 #include <iomanip>
4 using namespace std;
5
6 const double BASIC_FEE = 999.0;
7 const double SILVER_FEE = 1499.0;
8 const double GOLD_FEE = 1999.0;
9
10 class Member {
11 private:
12     int memberId;
13     string name;
14     string planType;
15     double fee;
16
17     static int activeMembers; // total active members
18 }
```

INPUT

```
3
101
Akhil
BASIC
102
Priya
STANDARD
103
Ravi
PREMIUM
```

Enter number of members:

--- Member 1 ---

Enter Member ID: Enter Name: Enter Plan Type (BASIC/SILVER/GOLD):

--- Member 2 ---

Enter Member ID: Enter Name: Enter Plan Type (BASIC/SILVER/GOLD):

--- Member 3 ---

Enter Member ID: Enter Name: Enter Plan Type (BASIC/SILVER/GOLD):

===== Membership Details =====

ID	Name	Plan	Fee
----	------	------	-----

Enter Member ID: Enter Name: Enter Plan Type (BASIC/SILVER/GOLD):

===== Membership Details =====

ID	Name	Plan	Fee
101	Akhil	BASIC	Fee: 999
102	Priya	INVALID	Fee: 0
103	Ravi	INVALID	Fee: 0

Total Active Members: 1

3. LOAN REPAYMENT SYSTEM

code:

```
#include <iostream>

#include<math>
using namespace std;

// ----- Inline function for simple interest -----
inline double simpleInterest(double principal, double rate, int years) {
return (principal * rate * years) / 100.0;
}

// ----- Function Prototypes (EMI-related) -----
double calculateEMI(double principal, double annualRate, int months);
double calculateInterestTotal(double principal, double annualRate, int months);
double applySubsidy(double payableAmount, double subsidyPercent, double& finalPayable);

// ----- Default arguments used where applicable -----
double calculateEMI(double principal, double annualRate = 10.0, int months = 12) {
// EMI formula:
//  $EMI = P * r * (1+r)^n / ((1+r)^n - 1)$ 
double r = (annualRate / 12.0) / 100.0; // monthly rate
int n = months;

if (r == 0) {
return principal / n;
}

double num = principal * r * (pow(1 + r, n));
double den = (pow(1 + r, n) - 1);
return num / den;
}

double calculateInterestTotal(double principal, double annualRate = 10.0, int months = 12) {
double emi = calculateEMI(principal, annualRate, months);
return (emi * months) - principal; // total interest paid
}

double applySubsidy(double payableAmount, double subsidyPercent = 15.0, double&
finalPayable) {
// call by reference to update payable amount after subsidy
double discount = (subsidyPercent / 100.0) * payableAmount;
```



```
finalPayable = payableAmount - discount;
return finalPayable;
}
```

```
int main() {
double loanAmount, annualInterestRate, subsidyPercent;
int months;
```

```
cout << "=== Loan Repayment System ===\n";
cout << "Enter Loan Amount: ";
cin >> loanAmount;
```

```
cout << "Enter Annual Interest Rate (in %). (default can be used if you want): ";
cin >> annualInterestRate;
```

```
cout << "Enter Tenure in Months: ";
cin >> months;
```

```
cout << "Enter Subsidy Percent (e.g., 15 for 15%): ";
cin >> subsidyPercent;
```

```
// EMI using function prototyping + default args (defaults not used here, but available)
double emi = calculateEMI(loanAmount, annualInterestRate, months);
```

```
// total interest
double totalInterest = calculateInterestTotal(loanAmount, annualInterestRate, months);
```

```
// total payable without subsidy
double payableAmount = emi * months;
```

```
// Show simple interest calculation using inline function (for clarity)
int years = months / 12;
double simpleInt = simpleInterest(loanAmount, annualInterestRate, years);
```

```
// Update final payable after subsidy (call by reference)
double finalPayable = 0.0;
applySubsidy(payableAmount, subsidyPercent, finalPayable);
```

```
cout << "\n--- Loan Repayment Details ---\n";
cout << "Loan Amount: " << loanAmount << "\n";
cout << "Annual Interest Rate: " << annualInterestRate << "%\n";
cout << "Tenure: " << months << " months\n";
```

```

cout << "Monthly EMI: " << emi << "\n";
cout << "Total Interest (EMI method): " << totalInterest << "\n";
cout << "Total Payable (without subsidy): " << payableAmount << "\n";

cout << "Simple Interest (inline example): " << simpleInt << "\n";
cout << "Subsidy Percent: " << subsidyPercent << "%\n";
cout << "Final Payable (after subsidy): " << finalPayable << "\n";

return 0;
}

```

output:

PROGRAM EDITOR

C++

Run

Save

```

1 #include <iostream>
2 #include <iomanip>
3 #include <cmath>
4 using namespace std;
5
6 // Function prototyping (A)
7 double calculateEMI(double principal, int years, double rate);
8 double calculateInterest(double principal, int years, double rate);
9 inline double simpleInterest(double principal, double rate); // D (inline)
10 void applySubsidy(double &payableAmount, double subsidyPercent); // C (call by referen
11
12 // Main function
13 int main() {
14     double loanAmount;
15     int years;
16     double interestRate; // interest rate in percentage
17     double subsidyPercent;
18
19     cout << "=== Loan Repayment System ===\n";

```

INPUT

```

1000000
3
0
15

```

```

=== Loan Repayment System ===
Enter Loan Amount: Enter Loan Duration (in years): Enter Interest Rate (in %).
(Press 0 to use default 8.5%): Enter Subsidy Percentage (e.g., 10 for 10%):
--- Loan Details ---
Loan Amount: 1000000.00
Interest Rate: 8.50%
Total Interest: 136431.35
Monthly EMI: 31567.54
Payable Amount after Subsidy: 965966.65

```

4. PARCEL CHARGE MANAGEMENT SYSTEM

Code:

```
#include <iostream>
#include <string>
using namespace std;

// Function prototyping (calculation functions inside class will be prototyped there)
class Parcel {
private:
    string parcelType;
    double weightKg;
    double distanceKm;
    double packagingCharge;

    // Static member to track total parcels processed
    static int totalParcels;

    // Inline function to calculate base charge
    inline double baseCharge(double w, double d) {
        // Base formula (can be adjusted): base rate per kg and per km
        return (w * 10.0) + (d * 5.0);
    }

public:
    // Constructor (optional)
    Parcel() : parcelType(""), weightKg(0), distanceKm(0), packagingCharge(0) {}

    // Default arguments for packaging charge
    void readDetails(const string& type, double w, double dist, double pack = 50.0) {
        parcelType = type;
        weightKg = w;
        distanceKm = dist;
        packagingCharge = pack;

        totalParcels++; // update static count
    }

    // Function overloading:
    // Normal Delivery
    double calculateCharge(double w, double d) {
        // Normal multiplier
```

```

double normalCost = baseCharge(w, d) * 1.0;
return normalCost + packagingCharge;
}

// Express Delivery (overloaded function)
double calculateCharge(double w, double d, bool express) {
if (express) {
// Express multiplier
double expressCost = baseCharge(w, d) * 1.5;
return expressCost + packagingCharge;
}
// If not express, treat as normal
return baseCharge(w, d) + packagingCharge;
}

void displayParcelInfo(double normalCharge, double expressCharge) {
cout << "\n--- Parcel Information ---\n";
cout << "Parcel Type: " << parcelType << endl;
cout << "Weight (kg): " << weightKg << endl;
cout << "Distance (km): " << distanceKm << endl;
cout << "Packaging Charge: " << packagingCharge << endl;

cout << "Normal Delivery Charge: " << normalCharge << endl;
cout << "Express Delivery Charge: " << expressCharge << endl;
}

// Static member function to display total parcels processed
static void displayTotalParcels() {
cout << "\nTotal Parcels Processed: " << totalParcels << endl;
}
};

// Initialize static data member
int Parcel::totalParcels = 0;

int main() {
int n;
cout << "Enter number of parcels: ";
cin >> n;

for (int i = 0; i < n; i++) {
Parcel p;
string type;

```

```

double w, dist;
double pack;

cout << "\nParcel " << (i + 1) << " Details:\n";
cout << "Enter parcel type: ";
cin >> type;
cout << "Enter weight (kg): ";
cin >> w;
cout << "Enter distance (km): ";
cin >> dist;

cout << "Enter packaging charge (enter -1 to use default=50): ";
cin >> pack;

if (pack == -1) {
    // default packaging charge used
    p.readDetails(type, w, dist);
} else {
    p.readDetails(type, w, dist, pack);
}

double normal = p.calculateCharge(w, dist);
double express = p.calculateCharge(w, dist, true);

p.displayParcelInfo(normal, express);
}

Parcel::displayTotalParcels();
return 0;
}

```

Output:

PROGRAM EDITOR

C++

Run

Save

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class Parcel {
6 private:
7     string parcelType;
8     double weightKg;
9     double distanceKm;
10    double packagingCharge;
11
12    // Static member to track total parcels processed
13    static int totalParcels;
14
15    // Inline function to calculate base charge
16    inline double baseCharge(double w, double d) {
17        // Base formula (can be adjusted): base rate per kg and per km
18        return (w * 10.0) + (d * 5.0);
19    }
20 }
```

INPUT

2
Normal
3
10
N
Express
5
20
Y
150

```
Enter number of parcels:
Parcel 1 Details:
Enter parcel type: Enter weight (kg): Enter distance (km): Enter packaging
charge (enter -1 to use default=50):
--- Parcel Information ---
Parcel Type: Normal
Weight (kg): 3
Distance (km): 10
Packaging Charge: 0
Normal Delivery Charge: 80
```

```
Parcel 2 Details:
Enter parcel type: Enter weight (kg): Enter distance (km): Enter packaging
charge (enter -1 to use default=50):
--- Parcel Information ---
Parcel Type:
Weight (kg): 3
Distance (km): 10
Packaging Charge: 0
Normal Delivery Charge: 80
Express Delivery Charge: 120
```

Express Delivery Charge: 120

Total Parcels Processed: 2

5.WATER BILLING SYSTEM

Code:

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

class Consumer {
private:
    int consumerId;
    string consumerType; // "Domestic" or "Commercial"
    double waterLiters; // consumption in liters
    double billAmount; // final bill (after tax/subsidy in required order)

    // Static member to maintain total consumers processed
    static int totalConsumers;

public:
    // Constructor
    Consumer() : consumerId(0), consumerType(""), waterLiters(0), billAmount(0) {}

    // a & b: Public functions to input details, calculate and display
    void inputDetails();
    void calculateBill();

    // h: display billing information
    void displayBilling() const;

    // Static function to display total consumers processed
    static void displayTotalConsumers();

    // c: Function overloading for Domestic and Commercial tariff rates
    // Each returns base bill without tax/subsidy
    double calculateBill(double liters, double domesticRate);
    double calculateBill(double liters, double commercialRate);

    // d: call by reference to update bill after subsidy
    void applySubsidy(double &amount, double subsidyPercent);

    // e: default arguments for service tax
```

```

// default service tax rate is used unless specified
void applyServiceTax(double &amount, double taxRate = 0.10);

// f: Function prototyping is already demonstrated by declaring above
};

// Initialize static member
int Consumer::totalConsumers = 0;

void Consumer::inputDetails() {
    cout << "\nEnter Consumer ID: ";
    cin >> consumerId;

    cout << "Enter Consumer Type (Domestic/Commercial): ";
    cin >> consumerType;

    cout << "Enter Water Consumption in Liters: ";
    cin >> waterLiters;

    // Validate consumer type
    if (consumerType != "Domestic" && consumerType != "Commercial") {
        cout << "Invalid type! Setting to Domestic by default.\n";
        consumerType = "Domestic";
    }

    totalConsumers++;
}

double Consumer::calculateBill(double liters, double domesticRate) {
    return liters * domesticRate;
}

double Consumer::calculateBill(double liters, double commercialRate) {
    return liters * commercialRate;
}

void Consumer::applySubsidy(double &amount, double subsidyPercent) {
    // subsidyPercent is percent value like 10 means 10%
    double subsidy = amount * (subsidyPercent / 100.0);
    amount -= subsidy;
}

void Consumer::applyServiceTax(double &amount, double taxRate) {

```



```
amount += amount * taxRate;
}
```

```
void Consumer::calculateBill() {
// Sample tariff rates (can be modified easily)
const double domesticRate = 0.05; // per liter
const double commercialRate = 0.08; // per liter

// Calculate base bill using overloading based on consumer type
if (consumerType == "Domestic") {
billAmount = calculateBill(waterLiters, domesticRate);
// Example: Domestic gets subsidy
applySubsidy(billAmount, 20); // 20% subsidy
} else {
billAmount = calculateBill(waterLiters, commercialRate);
// Example: Commercial gets smaller subsidy
applySubsidy(billAmount, 10); // 10% subsidy
}

// Apply service tax with default argument (can be overridden if needed)
applyServiceTax(billAmount); // uses default taxRate = 0.10
}
```

```
void Consumer::displayBilling() const {
cout << "\n===== Water Billing Details =====\n";
cout << "Consumer ID : " << consumerId << "\n";
cout << "Consumer Type : " << consumerType << "\n";
cout << "Water Consumption(L) : " << fixed << setprecision(2) << waterLiters << "\n";
cout << "Subsidy-adjusted Bill Amount: " << fixed << setprecision(2) << billAmount << "\n";
cout << "===== \n";
}
```

```
void Consumer::displayTotalConsumers() {
cout << "\nTotal Consumers Processed: " << totalConsumers << "\n";
}
```

```
int main() {
int n;
cout << "Enter number of consumers: ";
cin >> n;
```

```
if (n <= 0) {
cout << "Invalid number.\n";
```

```

return 0;
}

// h: Create an array of objects to store and process multiple consumers
Consumer consumers[100]; // max 100 for safety
if (n > 100) n = 100;

for (int i = 0; i < n; i++) {
    cout << "\n--- Consumer " << (i + 1) << " ---";
    consumers[i].inputDetails();
    consumers[i].calculateBill();
}

cout << "\n\n***** Displaying All Consumer Bills *****\n";
for (int i = 0; i < n; i++) {
    consumers[i].displayBilling();
}

Consumer::displayTotalConsumers();
return 0;
}

```

Output:

PROGRAM EDITOR

C++

Run

Save

```

1 #include <iostream>
2 #include <iomanip>
3 #include <string>
4 using namespace std;
5
6 class Consumer {
7 public:
8     // attributes
9     int consumerId;
10    string consumerType; // "Domestic" / "Commercial"
11    double waterLiters;
12    double billAmount;
13
14    // counters
15    static int totalConsumers;
16
17    // functions
18    Consumer() : consumerId(0), waterLiters(0), billAmount(0) {}
19

```

INPUT

```

3
101
Domestic
1200

102
Commercial
1800

103
Domestic
500

```

Enter number of consumers:

--- Consumer 1 ---

Enter Consumer Type (Domestic/Commercial): Enter Water Consumption (L):

--- Consumer 2 ---

Enter Consumer Type (Domestic/Commercial): Enter Water Consumption (L):

--- Consumer 3 ---

Enter Consumer Type (Domestic/Commercial): Enter Water Consumption (L):

***** Displaying All Consumer Bills *****

===== Water Billing Details =====

Consumer ID : 1000

Consumer Type : 101

Water Consumption(L) : 0.00

Subsidy-adjusted Bill Amount: 0.00

=====

===== Water Billing Details =====

Consumer ID : 1001

Consumer Type :

Water Consumption(L) : 0.00

Subsidy-adjusted Bill Amount: 0.00

=====

===== Water Billing Details =====

Consumer ID : 1002

Consumer Type :

Water Consumption(L) : 0.00

Subsidy-adjusted Bill Amount: 0.00

=====

Total Consumers Processed: 3